

Are the Database Engines Ready for Data Compliance Policies?

Ahana Pradhan
trovato@corporation.com
Institute for Clarity in Documentation
Dublin, Ohio, USA

Srinivas Karthik
The Thørvöld Group
Hekla, Iceland
larst@affiliation.org

Abstract

A clear and well-documented $\text{\LaTeX}$ document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the “acmart” document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

CCS Concepts

• **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

Ahana Pradhan and Srinivas Karthik. 2018. Are the Database Engines Ready for Data Compliance Policies?. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Data Compliance Regulatory Policies. : Modern data protection regulations such as the European Union’s General Data Protection Regulation (GDPR) impose strict requirements on how personal data may be accessed, processed, and disclosed [4]. Many GDPR obligations can be operationalized as *fine-grained* access constraints: only actors with an appropriate legal basis and context should be able to access particular records, and default system behavior should limit unnecessary access [1]. From a database systems perspective, these obligations often resemble *row-level* constraints, where access decisions depend on attributes of each tuple (e.g., data subject, tenant, jurisdiction, purpose, or organizational unit).

Compliance Policies as RLS. : Contemporary database vendors expose built-in *row-level security* (RLS) mechanisms that allow such constraints to be expressed as predicates and enforced transparently by the query engine [6, 10]. This vendor model aligns well with established database research on fine-grained access control

that enforces policies by rewriting or constraining queries so only authorized tuples contribute to results [11, 13]. In this sense, GDPR-like compliance policies can often be viewed as being enforced *in situ* by the DBMS: applications issue ordinary SQL, while the database injects (or semantically enforces) row predicates based on the session user, role, or application context.

Operational RLS. : One of the earliest database security mechanisms that directly corresponds to RLS is *access control by query modification*. Stonebraker and Wong describe rewriting queries so that authorization constraints become additional selection predicates, thereby restricting which tuples can contribute to the result [12]. This idea remains central: a policy is operationalized as a predicate (or a set of predicates) injected into query evaluation. In real systems, row-level access control has been adopted as a built-in enforcement mechanism. Database engines such as PostgreSQL and SQL Server provide *row security policies* that automatically append policy predicates to query plans, ensuring only authorized tuples are visible or modifiable [9, 7]. Oracle Database implements *Virtual Private Database (VPD)*, a fine-grained access control feature that dynamically attaches security predicates to SQL based on session and application context for the same purpose [8].

Pros of RLS for Compliance. : Using vendor-provided RLS as a compliance enforcement point has notable advantages. First, policy enforcement is centralized “close to the data,” reducing reliance on scattered application-side filtering and lowering the risk of inconsistent implementations across services [6, 10]. Second, RLS integrates with core DBMS functionality (transactions, views, and authorization), enabling a uniform control surface across many query forms [11]. Third, RLS can support operationalization efforts that translate regulatory requirements into enforceable technical controls and auditability workflows [2, 5].

Cons of RLS for Compliance. : However, recent literature [3] demonstrates that RLS implementation is not full-proof against side-channel attacks that can bypass data compliance regulation. They show that although RLS correctly filters unauthorized tuples from query results, the *execution time* of RLS-filtered queries can leak information about protected data via timing side channels. Their study demonstrates practical attacks on managed PostgreSQL and SQL Server instances where an adversary can infer unseen values or counts of sensitive tuples by crafting index-using queries and analyzing latency patterns.

Why a Taxonomy of Compliance Policies is Needed. Although row-level security (RLS) is widely used to enforce GDPR-like constraints, compliance policies are far from uniform. Even when expressed as SQL predicates, they vary substantially in structure—from single-table filters to multi-way joins, correlated subqueries, anti-joins (NOT EXISTS), and aggregation (GROUP BY/HAVING). This diversity

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

matters because policy structure directly influences the optimizer's choice of access paths and join strategies, leading to markedly different execution behavior even for policies with similar regulatory intent.

Query-graph Structure as a Base for Compliance Taxonomy. To reason about this diversity systematically, we define a taxonomy of compliance policies based on their *query-graph structure*. Beyond providing a compact vocabulary for describing real compliance rules, this structure also enables principled reasoning about enforcement fragility, since the same features strongly influence query plan selection and runtime cost.

Timing Side Channels Motivate Category-level Analysis. As noted before, query execution time can leak sensitive information even when RLS correctly filters unauthorized tuples from query outputs. Importantly, the feasibility and severity of such timing leakage depends on the *structure* of the enforced policy, since policy structure influences query planning and execution behavior. This motivates analyzing timing side-channel risk at the level of policy classes, rather than treating RLS as a monolithic enforcement mechanism.

Research Objective. Motivated by these considerations, we define a taxonomy of compliance policies grounded in query-graph structure and analyze the applicability of timing-based side-channel attacks for each category. This framing connects regulatory enforcement goals to DBMS execution mechanisms, and provides a systematic basis for understanding when RLS-based compliance is likely to leak sensitive information even when outputs are correctly filtered. Moreover, in practice, compliance is rarely enforced by a single rule. Deployments typically combine multiple policies to reflect layered business requirements (e.g., regional restrictions, retention limits, contractual scope, and role-based access), implemented as AND/OR compositions across different policy categories. These combinations are important because they can change the effective policy semantics in non-trivial ways: conjunctions may introduce stricter constraints than any individual rule, while disjunctions may introduce exceptions that broaden access. Moreover, combining heterogeneous policy structures (e.g., joins with aggregation or (anti-)existence constraints) often changes the resulting query graph and enforcement complexity. Therefore, a taxonomy grounded in query structure should account not only for individual policies, but also for how policies compose in realistic compliance deployments.

2 Contributions

This paper makes the following contributions:

- We introduce a taxonomy of data compliance and access-control policies grounded in query-graph structure. The taxonomy captures the dominant structural forms that arise in practice (e.g., local filters, relational joins, (anti-)existence constraints, aggregation-based rules, and correlated baselines), providing a compact vocabulary for describing real regulatory and business compliance requirements.
- For each policy category, we analyze the applicability of timing-based side-channel attacks in RLS-enforced execution. Our analysis connects policy structure to execution mechanisms (e.g., access paths, early termination, and intermediate result construction),

offering a systematic basis for understanding when RLS-based compliance can leak sensitive information even when query outputs are correctly filtered.

- Beyond single-policy enforcement, we consider combinations of multiple compliance policies composed via AND/OR across heterogeneous categories. We evaluate how robust industry-strength database engines are when enforcing such composed policies as RLS, highlighting the practical impact of policy interaction on execution behavior and leakage risk.

3 A Taxonomy of SQL Compliance Policies

We now present a taxonomy of data compliance and access-control policies expressed as SQL queries. The taxonomy covers and categorizes policies by their *semantic enforcement mechanism*. Each category below is illustrated using one representative policy, framed based on the TPC schema.

3.1 Local Row-level Filtering Policies

Definition. A local row-level filtering policy decides visibility of tuples using predicates over attributes of the same row, without requiring joins, subqueries, or aggregation. These policies encode basic compliance constraints such as retention windows, categorical allow/deny lists, numeric thresholds, and string-based filters.

Example (P1: retention window).

```
SELECT * FROM lineitem l WHERE l.shipdate >= CURRENT_DATE
- INTERVAL '5' YEAR;
```

3.2 Relational (Join-based) Filtering Policies

Definition. A relational filtering policy requires joining the target relation with one or more related relations, and constrains visibility using cross-entity attributes. These policies capture typical compliance rules involving geography, organizational structure, or role-based scoping across entities.

Example (P11: customer and supplier must share region).

```
SELECT l.* FROM lineitem l JOIN orders o ON
o.o_orderkey = l.l_orderkey JOIN customer c ON
c.c_custkey = o.o_custkey JOIN supplier s ON s.s_suppkey
= l.l_suppkey JOIN nation nc ON nc.n_nationkey =
c.c_nationkey JOIN nation ns ON ns.n_nationkey =
s.s_nationkey WHERE nc.n_regionkey = ns.n_regionkey;
```

3.3 Existence-based (Semi-join) Policies

Definition. An existence-based policy grants visibility to an entity if there exists at least one related record satisfying a qualifying condition. These policies encode common compliance notions such as “allow access if there is at least one valid transaction”.

Example (P12: orders visible if there exists a qualifying lineitem).

```
SELECT DISTINCT o.* FROM orders o JOIN lineitem l
ON l.l_orderkey = o.o_orderkey WHERE l.l_discount
BETWEEN 0.05 AND 0.10 AND l.l_extendedprice * (1
- l.l_discount) > 1000;
```

3.4 Universal / Violation-free (Anti-join) Policies

Definition. A universal policy enforces a *violation-free* constraint: an entity is visible only if there does not exist a related record that violates the policy. These policies naturally arise in regulatory settings that require “no exceptions” behavior, and are commonly expressed using NOT EXISTS.

Example (P28: orders visible only if no lineitem shipped late).

```
SELECT o.* FROM orders o WHERE NOT EXISTS (
SELECT 1 FROM lineitem l WHERE l.l_orderkey =
o.o_orderkey AND l.l_shipdate > l.l_commitdate );
```

3.5 Aggregation and Group-level Policies

Definition. Aggregation policies enforce compliance based on statistics computed over a group of tuples, such as counts, distinct counts, sums, averages, and extrema. These policies are typically expressed using GROUP BY and HAVING, and are common in threshold-style access control (e.g., minimum activity requirements).

Example (P23: customers visible if they have orders from ≥ 3 suppliers).

```
SELECT c.* FROM customer c JOIN orders o ON
o.o_custkey = c.c_custkey JOIN lineitem l ON l.l_orderkey
= o.o_orderkey GROUP BY c.c_custkey HAVING COUNT(DISTINCT
l.l_suppkey) >= 3;
```

3.6 Baseline-comparative Policies

Definition. Baseline-comparative policies constrain visibility based on how an entity compares against a peer-group baseline, such as “above the average of its category” or “below the global average”. These policies are often expressed using correlated scalar subqueries.

Example (P33: orders visible only if customer balance is above their nation average).

```
SELECT o.* FROM orders o JOIN customer c ON
c.c_custkey = o.o_custkey WHERE c.c_acctbal > (SELECT
AVG(c2.c_acctbal) FROM customer c2 WHERE c2.c_nationkey
= c.c_nationkey );
```

3.7 Proportionality Policies

Definition. Ratio policies enforce proportionality constraints, typically comparing an entity’s measure to a fraction or multiple of an aggregated baseline. Such policies are particularly relevant for compliance settings that encode dominance, exposure, or quota-based rules.

Example (P16: lineitem visible if it accounts for at least 20% of part quantity).

```
SELECT l.* FROM lineitem l WHERE l.l_quantity >=
0.2 * ( SELECT SUM(l2.l_quantity) FROM lineitem
l2 WHERE l2.l_partkey = l.l_partkey );
```

3.8 Statistical Dependence Policies

Definition. Statistical dependence policies impose constraints over higher-order statistics beyond classical aggregates. These include correlation, variance-like measures, and other statistical indicators. While less common in traditional access control, they capture advanced compliance requirements (e.g., detecting suspicious patterns).

Example (P50: orders visible only if discount correlates positively with quantity).

```
SELECT o.* FROM orders o JOIN lineitem l ON
l.l_orderkey = o.o_orderkey GROUP BY o.o_orderkey
HAVING CORR(l.l_quantity, l.l_discount) > 0;
```

3.9 Masking and Transformation (Column-level) Policies

Definition. Unlike the preceding categories, which enforce compliance by filtering tuples (row-level access control), masking policies enforce compliance by *transforming* sensitive attributes at query

Tag	Policies
L	P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P48, PII-3
J	P11, P12, P13, P14, P15, P16, P17, P18, P19, P20, P21, P22, P23, P24, P25, P26, P27, P28, P29, P30, P32, P33, P34, P37, P38, P39, P40, P41, P42, P43, P45, P46, P47, P48, P49, P50, PII-2, PII-3, PII-4, PII-5, PII-6, PII-7, PII-8, PII-10
E	P12, P15, P18, P19, P32, PII-1
U	P22, P28, P31, P39, P47, PII-5, PII-9
A	P17, P20, P21, P23, P24, P26, P27, P29, P34, P37, P38, P40, P41, P42, P43, P44, P46, P49, P50, PII-2, PII-7, PII-8, PII-10
C	P16, P25, P27, P33, P35, P36, P38, P40, P44
R	P16, P38
S	P50
M	PII-7

Table 1: Mapping of policies to taxonomy categories.

time. That is, the tuple may remain visible, but one or more protected fields are conditionally redacted or replaced with a safe surrogate. Such policies are common in PII governance, where coarse-grained row suppression is undesirable.

Example (PII-7: customer name is masked unless the customer has placed at least 3 orders).

```
SELECT c.c_custkey, CASE WHEN COUNT(o.o_orderkey)
>= 3 THEN c.c_name ELSE 'REDACTED' END AS c_name
FROM customer c LEFT JOIN orders o ON o.o_custkey
= c.c_custkey GROUP BY c.c_custkey, c.c_name;
```

4 Mapping Policies to Taxonomy Categories

We now map each policy in our suite to the taxonomy categories introduced in Section 3. Since many realistic compliance rules combine multiple enforcement mechanisms (e.g., joins with aggregation, or correlated baselines with ratios), we allow multi-label assignments. Table 1 presents the mapping in a tag-centric form.

Legend. **L**: local row filter; **J**: join-based relational filter; **E**: existence (semi-join); **U**: universal / violation-free (anti-join); **A**: aggregation / group-level constraint; **C**: baseline-comparative (correlated subquery); **R**: ratio / proportionality; **S**: statistical dependence; **M**: masking / transformation.

References

- [1] L. A. Bygrave. Data protection by design and by default: Deciphering the eu's legislative requirements. *Oslo Law Review*, 4(2):105–120, 2017.
- [2] T. R. Chhetri, A. Kurteva, R. J. DeLong, R. Hilscher, K. Korte, and A. Fensel. Data protection by design tool for automated gdpr compliance verification based on semantically modeled informed consent. *Sensors*, 22(7):2763, 2022.
- [3] C. Dar, M. Hershcovitch, and A. Morrison. Rls side channels: Investigating leakage of row-level security protected data through query execution time. *Proceedings of the ACM on Management of Data*, 1(1):1–25, 2023.
- [4] C. J. Hoofnagle, B. van der Sloot, and F. Zuiderveen Borgesius. The european union general data protection regulation: What it is and what it means. *Information & Communications Technology Law*, 28(1):65–98, 2019.
- [5] F. Kalinowski, D. Klein, M. Johns, and V. Moonsamy. Making web applications gdpr compliant: A comparative evaluation of gdpr-enforcement frameworks. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2025(4):777–794, 2025.
- [6] Microsoft. *Row-Level Security*, 2026. Microsoft Learn documentation for SQL Server Row-Level Security (versioned view parameter in URL).
- [7] Microsoft Corporation. *Row-Level Security*, 2025. Microsoft Learn documentation.
- [8] Oracle. Using Oracle Virtual Private Database to Control Data Access, 2025. Official Oracle documentation on Virtual Private Database (VPD): policies attach predicates to tables, enforcing row/column access control.
- [9] PostgreSQL Global Development Group. *Documentation: Row Security Policies*, 2025. Official documentation describing PostgreSQL RLS implementation and semantics.
- [10] PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies*, 2026. Online documentation for Row-Level Security semantics (current branch).
- [11] S. J. Rizvi, A. O. Mendelzon, S. Sudarshan, and P. Roy. Extending query rewriting techniques for fine-grained access control. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, pages 551–562. ACM, 2004.
- [12] M. Stonebraker and E. Wong. Access control in a relational data base management system by query modification. In *Proceedings of the 1974 ACM Annual Conference*, pages 180–186. ACM, 1974.
- [13] Q. Wang, T. Yu, N. Li, J. Lobo, E. Bertino, K. Irwin, and J.-W. Byun. On the correctness criteria of fine-grained access control in relational databases. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*, pages 555–566, 2007. Proceedings DOI minted by ACM for the proceedings entry.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009